

CO5_ASSESSMENTS 2

PSEUDOCODE WRITING TASK

Name : Krishna Tejesh Reddy . Y

Reg no : 192411200

Course : Computer Vision

1. Real-Time Face Detection and Recognition

Aim

To design an OpenCV-based real-time face detection and recognition system that captures live video, detects multiple faces, recognizes registered persons, and displays bounding boxes and identity labels.

Requirements

The system should:

- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.
- Minimize processing delay.
- Handle multiple faces simultaneously.

Pseudocode

START

IMPORT OpenCV

IMPORT required face recognition libraries

LOAD trained face recognition dataset

LOAD face detection model
LOAD trained recognition model

INITIALIZE camera

IF camera is not available THEN
 DISPLAY "Camera not found"
 STOP
END IF

WHILE camera is running

 CAPTURE a frame from camera

 IF frame is empty THEN
 BREAK
 END IF

 RESIZE frame to a smaller resolution
 CONVERT frame to grayscale
 APPLY noise reduction if required

 DETECT all faces in the frame

 FOR each detected face

 GET face bounding box coordinates

 EXTRACT face region from the frame

 RESIZE face region to required size
 NORMALIZE the face image

 EXTRACT facial features

 COMPARE extracted features
 WITH features stored in trained dataset

 IF matching score is above recognition threshold THEN
 IDENTIFY the registered person

```
    ELSE
        LABEL person as "Unknown"
    END IF

    DRAW bounding box around detected face
    DISPLAY identity label above bounding box

END FOR

DISPLAY processed frame

IF user presses "Q" THEN
    BREAK
END IF

END WHILE

RELEASE camera
CLOSE all OpenCV windows

STOP
```

Explanation of the Pseudocode

1. Initialize the system:

OpenCV and the required face-recognition components are loaded. The trained dataset and recognition model are also loaded.

2. Capture video:

The camera continuously captures live frames.

3. Preprocessing:

Each frame is resized to reduce computation. Grayscale conversion and noise reduction can be applied to improve detection and reduce processing time.

4. Face Detection:

The system detects all faces present in each frame and obtains their bounding-box coordinates.

5. Feature Extraction:

The detected face is cropped, resized, normalized, and converted into a feature representation.

6. Recognition:

The extracted features are compared with the trained dataset. If the similarity exceeds the selected threshold, the person's identity is displayed. Otherwise, the person is labeled as **Unknown**.

7. Visualization:

Bounding boxes are drawn around detected faces, and identity labels are displayed.

8. Efficient Loop Control:

The frame can be resized before processing to reduce computation. The loop continues until the camera stops or the user presses the **Q** key.

Conclusion

The pseudocode provides a complete real-time face detection and recognition pipeline. It handles multiple faces simultaneously and uses preprocessing, feature extraction, recognition thresholds, and efficient frame processing to minimize delay.

2. Vehicle Detection, Tracking and Counting

Aim

To design an OpenCV-based system that detects vehicles entering a predefined region, tracks multiple vehicles, counts vehicles crossing a virtual line, and displays tracking information in real time.

Requirements

The system should:

- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.

- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.
- Avoid double counting.
- Maintain near real-time performance.

Pseudocode

START

IMPORT OpenCV

IMPORT required vehicle detection and tracking libraries

INITIALIZE video camera or video file

DEFINE Region of Interest (ROI)

DEFINE virtual counting line

INITIALIZE vehicle detector

INITIALIZE vehicle tracker

CREATE empty set of counted vehicle IDs

SET vehicle_count = 0

IF video source is not available THEN

 DISPLAY "Video source not found"

 STOP

END IF

WHILE video is running

 CAPTURE current frame

 IF frame is empty THEN

 BREAK

 END IF

 RESIZE frame

 APPLY preprocessing

REDUCE noise if required

DETECT vehicles inside the ROI

FOR each detected vehicle

 GET bounding box coordinates

 CALCULATE vehicle center point

END FOR

SEND detected vehicles to tracker

UPDATE tracking information

FOR each tracked vehicle

 ASSIGN or maintain unique tracking ID

 GET current center position

 GET previous center position

 DRAW bounding box

 DISPLAY tracking ID

 IF vehicle crosses the virtual line THEN

 IF vehicle ID is NOT present

 in counted vehicle IDs THEN

 INCREASE vehicle_count by 1

 ADD vehicle ID to counted vehicle IDs

 END IF

 END IF

END FOR

```
    DISPLAY virtual counting line
    DISPLAY vehicle count
    DISPLAY tracking IDs
    DISPLAY processed frame

    IF user presses "Q" THEN
        BREAK
    END IF

END WHILE

RELEASE video source
CLOSE all OpenCV windows

DISPLAY final vehicle count

STOP
```

Explanation of the Pseudocode

1. Video Acquisition:

The system initializes a live camera or video file and continuously captures frames.

2. Region of Interest:

A predefined ROI is selected so that only the relevant road area is processed. This reduces unnecessary computation.

3. Preprocessing:

The input frame can be resized and filtered to reduce noise and improve detection performance.

4. Vehicle Detection:

The system detects vehicles present inside the selected ROI and obtains their bounding boxes.

5. Vehicle Tracking:

Each detected vehicle is assigned a unique tracking ID. The tracker maintains the identity of vehicles across consecutive frames.

6. Center Point Calculation:

The center point of each bounding box is calculated. The previous and current center positions are compared to determine movement.

7. Line Crossing:

A virtual line is drawn across the road. When a tracked vehicle moves across this line, the system checks its tracking ID.

8. Avoiding Double Counting:

Before increasing the vehicle count, the system checks whether the vehicle's ID already exists in the counted vehicle IDs set.

For example:

```
IF vehicle ID NOT IN counted vehicle IDs
    vehicle_count = vehicle_count + 1
    ADD vehicle ID to counted vehicle IDs
END IF
```

This ensures that the same vehicle is counted only once.

9. Visualization:

The system displays the vehicle bounding boxes, tracking IDs, virtual counting line, and total vehicle count.

10. Efficient Loop Termination:

The system continues processing frames until the video ends or the user presses **Q**. Frames can be resized and only the required ROI can be processed to maintain near real-time performance.

Conclusion

The proposed pseudocode provides a complete vehicle detection, tracking, and counting system. Unique tracking IDs and a set of counted IDs prevent double counting, while ROI processing and frame resizing help maintain near real-time performance. The system can simultaneously track multiple vehicles and display their IDs and total count.